

Il piano tecnico per la versione 2.0 (Fase 1: SQLite)

Nota: le immagini iniziali compaiono insieme al primo progetto, ma corrispondono alla cartella ``auditoria_version_2_0`` indicata all'inizio del documento relativo alla versione 2.0.

Aggiungeremo un file chiamato `core/database.py`. Ciò ci consentirà di salvare i risultati in modo strutturato.

Valutazione dell'ambiente tecnico

Prima di procedere con l'implementazione del codice, è stata verificata la corretta installazione delle dipendenze necessarie nell'ambiente di sviluppo (Kali Linux):

Attualmente, lo script è "effimero" (o **stateless**). Al termine della scansione, i risultati vengono visualizzati a schermo e vanno persi definitivamente.

Grazie all'integrazione di SQLite, stiamo trasformando lo strumento in un'applicazione **stateful**. Ciò significa che il programma ora possiede una "memoria":

1. Può salvare i risultati.
2. Può confrontare i risultati passati con quelli attuali (monitorando l'evoluzione della sicurezza).
3. Crea un database locale interrogabile in futuro.

Lo schema dei dati (la tabella `audit_results`)

Per la sezione 6.1 del report, abbiamo definito una struttura standard nell'ambito dell'attività di audit. A ogni esecuzione di una scansione, viene salvato un log contenente i seguenti dati:

- **id:** Un numero univoco per ciascuna voce.
- **target:** Il dominio o l'indirizzo IP oggetto di analisi.
- **test_name:** Il tipo di test eseguito (ad es. "Header Check").
- **status:** l'esito del test (superato o fallito).
- **score:** Una valutazione numerica (KPI).
- **timestamp:** *Data e ora esatte. Elemento fondamentale per mantenere uno storico.*

L'implementazione dettagliata (`core/database.py`)

Ecco il codice, ma con commenti esplicativi riga per riga in modo da poter capire cosa succede in ogni parte:

- Questo modulo `core/database.py` Implementa un database locale (SQLite) per trasformare il tuo scanner in uno strumento dotato di memoria. Utilizzando le funzioni `init_db` y `save_result`, il codice automatizza la memorizzazione storica dei risultati (target, stato, punteggio e data), consentendo alla logica di scansione di rimanere indipendente dal livello dati. Questa architettura migliora la tracciabilità dei report e predispone il sistema per future funzionalità di monitoraggio continuo.

Nota: nella libreria sqlite3, l'oggetto utilizzato per eseguire i comandi SQL è denominato **cursor**.

```
GNU nano 8.4 core/database.py
import sqlite3

# 1. Funzione di preparazione (Inizializzazione)
def init_db():
    # Connessione al file .db (creato automaticamente se inesistente)
    conn = sqlite3.connect('scanner_history.db')
    cursor = conn.cursor()

    # Creazione della table se non già esistente (IF NOT EXISTS)
    cursor.execute('''
        CREATE TABLE IF NOT EXISTS audit_results(
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            target TEXT,
            test_name TEXT,
            status TEXT,
            score REAL,
            timestamp DATETIME DEFAULT CURRENT_TIMESTAMP
        )
    ''')
    conn.commit() # Salvataggio delle modifiche
    conn.close() # Chiusura della connessione per evitare di lasciare thread aperti

# 2. Funzione di registrazione dati (Persistenza)
def save_result(target, result):
    conn = sqlite3.connect('scanner_history.db')
    cursor = conn.cursor()

    # Recuperiamo il valore 'score'; se la chiave manca, impostiamo 0 come default
    # Questo evita il 'KeyError' quando il modulo CMS non restituisce un punteggio
    score_val = result.get('score', 0)

    # Inserimento dei dati ricevuti
    cursor.execute('''
        INSERT INTO audit_results (target, test_name, status, score)
        VALUES (?, ?, ?, ?)
    ''', (target, result['test'], result['status'], score_val))
    conn.commit() # Conferma dell'operazione di scrittura sul file
    conn.close()
```

- Importiamo la funzione dal file `core/database.py` che abbiamo appena creato.

```
GNU nano 8.4 core/scanner.py
import socket
import ssl
import requests
import time

from core.utils import get_severity, calculate_header_score, check_dns_record

# Importa il modulo di rilevamento passivo del CMS.
# Utilizza la metodologia di analisi non invasiva di WPScan per identificarle le tecnologie web senza generare traffico
from core.cms_detector import CMSDetector

from core.database import save_result # Importiamo la funzione da database.py
```

- Questa modifica a `run_audit` ottimizza il flusso di esecuzione utilizzando un ciclo che richiama `save_result` subito dopo ogni test. Tale intervento non solo riduce la ridondanza del codice, ma assicura anche una **persistenza atomica**: ogni risultato viene scritto nel file `.db` non appena viene generato, garantendo così che i dati non vadano persi anche in caso di interruzione imprevista del programma durante la scansione.

```
def run_audit(self):
    """Esegue tutti i test e restituisce l'elenco dei risultati"""
    audit_data = []

    # Eseguiamo ogni test e salviamo
    for scan_func in [self.scan_dns(), self.scan_tls(), self.scan_headers(),
                     self.scan_sensitive_files(), self.scan_bot_ai_behavior()]:
        # Queste due righe DEVONO avere la rientranza (indentazione)
        audit_data.append(scan_func)
        save_result(self.target, scan_func)

    # Scansione del CMS
    detector = CMSDetector(self.target)
    cms_result = detector.run_passive_scan()
    audit_data.append(cms_result)
    save_result(self.target, cms_result)

    return audit_data
```

- Dobbiamo inserire l'inizializzazione all'inizio della funzione `main()`. **¿Perché?** Perché dobbiamo assicurarci che il file del database e la struttura della tabella esistano prima che lo

scanner tenti di scrivere dei risultati. Se lo scanner provasse a salvare dati in una tabella non ancora creata, il programma fallirebbe.

```
GNU nano 8.4 main.py
import argparse
from core.scanner import SecurityScanner
from reports.generator import ReportGenerator

# Importiamo la funzione per inizializzare il DB
from core.database import init_db

def main():
    # Inizializzazione del database: crea la struttura se non esiste
    init_db()
```

- Ogni riga visualizzata nel terminale rappresenta un record univoco nel database SQLite. Ecco il significato di ciascuna colonna:
 - **Colonna 1 (ID):** L'identificatore univoco della transazione (a incremento automatico).
 - **Colonna 2 (Target):** Il dominio sottoposto ad audit (asociaciondigitalcanaria.es).
 - **Colonna 3 (Test):** Il tipo di controllo eseguito (ad es. sicurezza DNS, TLS/SSL).
 - **Colonna 4 (Status):** Il risultato ([PASS](#), [FAIL](#), [PLANNED](#)).
 - **Colonna 5 (Score):** Il punteggio numerico (assegnato secondo la logica impostata, ad esempio 0,0 o 10,0).
 - **Colonna 6 (Timestamp):** L'orario esatto in cui è stato eseguito l'audit.
- **Aspetto fondamentale:** notiamo record con timestamp diversi (ad esempio, 09:19:21 e 09:24:16). Ciò dimostra che il database sta salvando i progressi nel tempo. Ora disponiamo di una cronologia degli audit!

```
(venv)-(guillermo@vbox)-[~/proyecto_seguridad/auditoria_seguridad]
$ python3 main.py --url https://asociaciondigitalcanaria.es
[*] Iniciando framework de auditoria para: https://asociaciondigitalcanaria.es
[DEBUG] Formate recibido: 'md'
[+] Report Markdown generado: report.md
[DEBUG] Formate recibido: 'json'
[+] Report JSON generado: report.json
[+] Auditoria finalizada exitosamente.
[+] Reporte generado en formato: MD

(venv)-(guillermo@vbox)-[~/proyecto_seguridad/auditoria_seguridad]
$ sqlite3 scanner_history.db "SELECT * FROM audit_results;"
1|asociaciondigitalcanaria.es|DNS Security (SPF/DMARC)|FAIL|0.0|2026-07-31 09:19:21
2|asociaciondigitalcanaria.es|TLS/SSL|PASS|10.0|2026-07-31 09:19:21
3|asociaciondigitalcanaria.es|Security Headers|PASS|8.3|2026-07-31 09:19:21
4|asociaciondigitalcanaria.es|Sensitive Files|PLANNED|0.0|2026-07-31 09:19:21
5|asociaciondigitalcanaria.es|Bot/IA Detection|PLANNED|0.0|2026-07-31 09:19:21
6|asociaciondigitalcanaria.es|DNS Security (SPF/DMARC)|FAIL|0.0|2026-07-31 09:24:16
7|asociaciondigitalcanaria.es|TLS/SSL|PASS|10.0|2026-07-31 09:24:16
8|asociaciondigitalcanaria.es|Security Headers|PASS|8.3|2026-07-31 09:24:16
9|asociaciondigitalcanaria.es|Sensitive Files|PLANNED|0.0|2026-07-31 09:24:16
10|asociaciondigitalcanaria.es|Bot/IA Detection|PLANNED|0.0|2026-07-31 09:24:16
11|asociaciondigitalcanaria.es|CMS Fingerprinting|PASS|0.0|2026-07-31 09:24:16
```

GESTIONE E FILTRAGGIO DELLA CRONOLOGIA

- **Query SQL:** Utilizziamo `ORDER BY timestamp DESC` affinché le scansioni più recenti appaiano per prime e `LIMIT 10` per evitare di affollare la schermata, visualizzando solo le ultime dieci voci.

- **fetchall():** Questo metodo recupera tutte le righe restituite dalla query e le converte in una lista di tuple (dove ogni riga è rappresentata come una tupla).
- **Ciclo for:** Iteriamo sulla lista (`rows`) e stampiamo ogni elemento in modo formattato, utilizzando gli indici corretti della tabella (dove `row[5]` è la data, `row[2]` è il nome del test, ecc.).

```
def get_history():
    # Connessione al database
    conn = sqlite3.connect('scanner_history.db')
    cursor = conn.cursor()

    # Recuperiamo tutti i record
    cursor.execute("SELECT * FROM audit_results ORDER BY timestamp DESC LIMIT 10")
    rows = cursor.fetchall()

    print("\n— Ultimi 10 Audit eseguiti —")
    for row in rows:
        print(f"[{row[5]}] {row[2]} | {row[3]} | Punteggio: {row[4]}")

    conn.close()
```

- Cosa fa questa modifica?
 - **action="store_true":** Indica a Python che, se l'utente digita `--history`, la variabile assumerà il valore `True`. Se non viene digitato nulla, il valore sarà `False`.
 - **if args.history:** All'inizio di `main()`, verifichiamo se l'utente desidera visualizzare la cronologia. In tal caso, richiamiamo la tua nuova funzione e utilizziamo `return` per terminare immediatamente il programma, evitando così che tenti di eseguire un audit senza un `URL`.

```
GNU nano 8.4 main.py
# Importiamo la funzione per inizializzare il DB e get_history
from core.database import init_db, get_history

def main():
    # Inizializzazione del database: crea la struttura se non esiste
    init_db()

    # Configuración de la interfaz CLI
    parser = argparse.ArgumentParser(description="Website Security Baseline Audit Tool - Framework de auditoria pasiva")

    # Argumentos que aceptará la herramienta
    parser.add_argument("--url", required=False) # url non obbligatoria se usiamo --history
    parser.add_argument("--format", choices=["json", "md"], default="md", help="Formato de reporte (json o md)")

    # Aggungli il nuovo flag
    parser.add_argument("--history", action="store_true", help="Mostra lo storico degli audit")
    # Aqui creamos args
    args = parser.parse_args()

    # 3. Logica per gestire l'argomento --history
    if args.history:
        get_history()
        return # Termina il programa qui dopo aver mostrato la storia

    if not args.url:
        print("Errore: Devi fornire una --url o usare --history")
        return
```

- Questo output mostra l'esecuzione corretta della tua nuova funzione `get_history()`. Ecco una spiegazione di ciò che il programma svolge "dietro le quinte": Interrogazione del database: Il programma ha aperto il file `scanner_history.db` ed eseguito una query `SELECT` per recuperare i dati dalla tabella `audit_results`. Ordinamento cronologico: Grazie all'istruzione `ORDER BY timestamp DESC`, il sistema ha preso i record più recenti (quelli con timestamp 09:24:16) e li ha posizionati in cima, spostando più in basso quelli meno recenti (09:19:21). Limitazione dei risultati: L'istruzione `LIMIT 10` garantisce che, anche se il database dovesse arrivare a contenere migliaia di record, lo schermo non venga sovraccaricato di informazioni superflue; vengono visualizzati solo i dati rilevanti (le ultime 10 voci).

- **Formateo:** El bucle for en tu código ha tomado cada "fila" de la base de datos y la ha transformado en una línea de texto legible para humanos, añadiendo los corchetes y las barras separadoras que ves.

```
(venv)-(guillermo@vbox)-[~/proyecto_seguridad/auditoria_seguridad]
$ python3 main.py --history

— Ultimi 10 Audit eseguiti —
[2026-07-31 09:24:16] DNS Security (SPF/DMARC) | FAIL | Punteggio: 0.0
[2026-07-31 09:24:16] TLS/SSL | PASS | Punteggio: 10.0
[2026-07-31 09:24:16] Security Headers | PASS | Punteggio: 8.3
[2026-07-31 09:24:16] Sensitive Files | PLANNED | Punteggio: 0.0
[2026-07-31 09:24:16] Bot/IA Detection | PLANNED | Punteggio: 0.0
[2026-07-31 09:24:16] CMS Fingerprinting | PASS | Punteggio: 0.0
[2026-07-31 09:19:21] DNS Security (SPF/DMARC) | FAIL | Punteggio: 0.0
[2026-07-31 09:19:21] TLS/SSL | PASS | Punteggio: 10.0
[2026-07-31 09:19:21] Security Headers | PASS | Punteggio: 8.3
[2026-07-31 09:19:21] Sensitive Files | PLANNED | Punteggio: 0.0
```

- Spiegazione del codice:
 - **Query dinamica con LIKE:** Invece di un confronto per corrispondenza esatta (=), utilizziamo **LIKE ?**. Il simbolo ? è un parametro di sicurezza che previene la SQL injection, mentre **LIKE** consente di cercare corrispondenze parziali anziché esatte.
 - **Pulizia dell'URL (Pre-elaborazione):** Prima di eseguire la query SQL, puliamo l'input dell'utente utilizzando `.replace()` e `.rstrip()`. Rimuoviamo protocolli come `https://` o `http://` e le barre finali (/). Questo passaggio è fondamentale affinché il testo cercato tramite SQL corrisponda al formato "pulito" (il dominio puro) memorizzato nel database.
 - **La potenza del carattere jolly %:** Racchiudendo l'URL pulito tra i simboli % (ad es. `'%'+clean_url+'%'`), indichiamo a SQL di eseguire una ricerca flessibile. Ciò consente al motore del database di individuare il record anche se l'utente ha inserito solo una parte dell'URL o se sono presenti lievi variazioni.
 - **Flessibilità:** Aggiungendo `target_url=None` come argomento opzionale, la funzione rimane pienamente compatibile con le chiamate esistenti (che non prevedevano filtri), pur risultando molto più potente grazie alla possibilità di applicare filtri specifici su richiesta.

```
def get_history(target_url=None):
    # Connessione al database
    conn = sqlite3.connect('scanner_history.db')
    cursor = conn.cursor()

    # Se viene fornita una URL, filtriame per essa, altrimenti mostriamo tutto
    if target_url:
        # Puliamo l'URL: rimuoviamo il protocollo affinché corrisponda a quello del database
        clean_url = target_url.replace("https://", "").replace("http://", "").rstrip('/')
        # Ora cerchiamo solo la parte pulita del dominio
        query = "SELECT * FROM audit_results WHERE target LIKE ? ORDER BY timestamp DESC LIMIT 10"
        cursor.execute(query, ('%' + clean_url + '%'))
    else:
        query = "SELECT * FROM audit_results ORDER BY timestamp DESC LIMIT 10"
        cursor.execute(query)
        rows = cursor.fetchall()

    print(f"\n— Storico Audit {target_url or ''} —")
    if not rows:
        print("Nessun risultato trovato per questo target.")
    for row in rows:
        print(f"[{row[5]}] {row[2]} | {row[3]} | Punteggio: {row[4]}")

    conn.close()
```

- Questa sezione gestisce il flusso di esecuzione in base ai comandi dell'utente: innanzitutto, se rileva l'argomento `--history`, visualizza la cronologia di audit e interrompe il programma per evitare di eseguire una nuova scansione. Successivamente, verifica la presenza di un URL; in caso di assenza, termina l'esecuzione e mostra un messaggio di errore per prevenire malfunzionamenti dovuti alla mancanza di un obiettivo.

```
# 3. Logica per gestire l'argomento --history
if args.history:
    get_history(args.url) # Passiamo l'URL inserito dall'utente a --url
    return # Termina il programma qui dopo aver mostrato la storia

if not args.url:
    print("Errore: Devi fornire una --url o usare --history")
    return
```

- Spiegazione del motivo per cui si ottiene lo stesso output per entrambi i comandi. Il fatto che l'output sia identico potrebbe destare qualche dubbio, ma esiste una spiegazione logica e positiva:
 - **Il database contiene un solo dominio:** il database `scanner\_history.db` attualmente conserva record relativi esclusivamente a `asociaciondigitalcanaria.es`.
 - **Logica di filtraggio:** Quando si esegue `python3 main.py --history`, il sistema recupera tutti i record (che appartengono tutti a quel dominio). Quando si esegue `python3 main.py --history --url ...`, il sistema applica il filtro `LIKE` e recupera tutti i record corrispondenti a quel dominio (il che, ancora una volta, include tutti i record presenti).
 - **Conclusione:** il fatto che entrambi i comandi producano lo stesso risultato conferma che il filtro funziona correttamente; se il filtro non funzionasse o fosse configurato in modo errato, il secondo comando restituirebbe "Nessun risultato trovato".

```
(venv)-(guillermo@vbox)-[~/proyecto_seguridad/auditoria_seguridad]
$ python3 main.py --history

--- Storico Audit per ---
[2026-07-31 09:24:16] DNS Security (SPF/DMARC) | FAIL | Punteggio: 0.0
[2026-07-31 09:24:16] TLS/SSL | PASS | Punteggio: 10.0
[2026-07-31 09:24:16] Security Headers | PASS | Punteggio: 8.3
[2026-07-31 09:24:16] Sensitive Files | PLANNED | Punteggio: 0.0
[2026-07-31 09:24:16] Bot/IA Detection | PLANNED | Punteggio: 0.0
[2026-07-31 09:24:16] CMS Fingerprinting | PASS | Punteggio: 0.0
[2026-07-31 09:19:21] DNS Security (SPF/DMARC) | FAIL | Punteggio: 0.0
[2026-07-31 09:19:21] TLS/SSL | PASS | Punteggio: 10.0
[2026-07-31 09:19:21] Security Headers | PASS | Punteggio: 8.3
[2026-07-31 09:19:21] Sensitive Files | PLANNED | Punteggio: 0.0

(venv)-(guillermo@vbox)-[~/proyecto_seguridad/auditoria_seguridad]
$ python3 main.py --history --url https://asociaciondigitalcanaria.es

--- Storico Audit per https://asociaciondigitalcanaria.es ---
[2026-07-31 09:24:16] DNS Security (SPF/DMARC) | FAIL | Punteggio: 0.0
[2026-07-31 09:24:16] TLS/SSL | PASS | Punteggio: 10.0
[2026-07-31 09:24:16] Security Headers | PASS | Punteggio: 8.3
[2026-07-31 09:24:16] Sensitive Files | PLANNED | Punteggio: 0.0
[2026-07-31 09:24:16] Bot/IA Detection | PLANNED | Punteggio: 0.0
[2026-07-31 09:24:16] CMS Fingerprinting | PASS | Punteggio: 0.0
[2026-07-31 09:19:21] DNS Security (SPF/DMARC) | FAIL | Punteggio: 0.0
[2026-07-31 09:19:21] TLS/SSL | PASS | Punteggio: 10.0
[2026-07-31 09:19:21] Security Headers | PASS | Punteggio: 8.3
[2026-07-31 09:19:21] Sensitive Files | PLANNED | Punteggio: 0.0
```

GESTIONE E MANUTENZIONE DELLA CRONOLOGIA

- Inoltre, la funzione `clear\_history` gestisce la manutenzione del database SQLite. Questa funzione stabilisce una connessione al file `scanner\_history.db`, esegue un'istruzione SQL per eliminare tutti i record dalla tabella `audit\_results`, salva definitivamente le modifiche tramite un'operazione di commit e — dopo aver chiuso la connessione — notifica all'utente che la cronologia è stata cancellata con successo.

```
def clear_history():
    conn = sqlite3.connect('scanner_history.db')
    cursor = conn.cursor()

    # Elimina tutti i record dalla tabella
    cursor.execute("DELETE FROM audit_results")

    # Conferma le modifiche
    conn.commit()
    conn.close()
    print("\n[+] Cronologia di audit eliminata correttamente.")
```

- Questa sezione iniziale dello script importa tutte le dipendenze necessarie al funzionamento del programma. Include la libreria `argparse` per la gestione dei comandi dell'utente; le classi `SecurityScanner` e `ReportGenerator`, che eseguono la logica di audit e la generazione dei report; e, infine, le funzioni per il database (`init_db`, `get_history`, `clear_history`) che consentono di memorizzare, interrogare e cancellare lo storico dell'audit.

```
GNU nano 8.4 main.py
import argparse
from core.scanner import SecurityScanner
from reports.generator import ReportGenerator

# Importiamo la funzione per inizializzare il DB, get_history e clear_history
from core.database import init_db, get_history, clear_history
```

- Ecco un riepilogo di questa sezione:
 - Questa parte configura gli argomenti della riga di comando aggiungendo i flag `--history` e `--clear_history`. Successivamente, il codice elabora l'intento dell'utente:
 - Pulizia:** se l'utente attiva `--clear_history`, il sistema richiede una conferma manuale e, in caso di conferma, esegue la funzione di eliminazione del database.
 - Query:** se viene rilevato il flag `--history`, il programma chiama `get_history` per visualizzare i record memorizzati e termina immediatamente l'esecuzione.

```
# Aggiungli il nuovo flag
parser.add_argument("--history", action="store_true", help="Mostra lo storico degli audit")
parser.add_argument("--clear_history", action="store_true", help="Eliminar l'intera cronologia di audit")
# Aqui creamos args
args = parser.parse_args()

# 3. — Gestione e Manutenzione della Cronologia —
if args.clear_history:
    confirm = input("Sei sicuro di voler eliminare l'intera cronologia? (s/n)")
    if confirm.lower() == 's':
        clear_history()
    else:
        print("Operazione annullata.")
    return

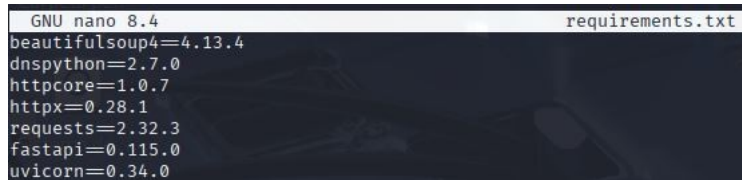
# 4. — Gestione e Filtraggio della Cronologia —
if args.history:
    get_history(args.url)
    return
```

- Questo screenshot documenta l'esecuzione del comando `--clear_history`, evidenziando la misura di sicurezza integrata: una richiesta di conferma manuale prima di procedere con l'eliminazione. Rispondendo con 's', l'utente conferma la propria intenzione; il sistema esegue quindi in sicurezza la procedura di pulizia del database e restituisce una notifica di conferma, garantendo così che i dati storici non vengano cancellati accidentalmente.

```
(venv)-(guillermo@vbox)-[~/proyecto_seguridad/auditoria_seguridad]
$ python3 main.py --clear_history
Sei sicuro di voler eliminar l'intera cronologia? (s/n)s

[+] Cronologia di audit eliminata correttamente.
```


- Il file `requirements.txt` definisce le dipendenze essenziali del progetto, garantendo che l'ambiente di esecuzione disponga di tutte le librerie necessarie. Questa versione include ``fastapi`` e ``uvicorn`` per abilitare le funzionalità di web API, oltre a librerie fondamentali come ``requests``, ``httpx``, ``beautifulsoup4`` e ``dnspython``, che supportano la logica centrale di scansione e di audit della sicurezza.
- Nota: ho aggiunto ``uvicorn`` (il server ASGI necessario per eseguire FastAPI) e ``fastapi``.



```

GNU nano 8.4 requirements.txt
beautifulsoup4=4.13.4
dnspython=2.7.0
httpcore=1.0.7
httpx=0.28.1
requests=2.32.3
fastapi=0.115.0
uvicorn=0.34.0

```

- Questo screenshot documenta la risoluzione dell'errore ``externally-managed-environment``, una misura di sicurezza presente in sistemi come Kali Linux che impedisce l'installazione di pacchetti Python a livello di sistema per evitare conflitti di compatibilità.
- Per risolvere il problema, è stato innanzitutto messo in sicurezza lo stato del progetto rinominando l'ambiente virtuale corrotto in ``venv_antiguo``, così da prevenire la perdita di dati. Successivamente, è stato creato un ambiente pulito utilizzando il comando ``python3 -m venv venv`` e attivato tramite ``source venv/bin/activate``, ripristinando così la possibilità di installare le dipendenze del progetto in modo isolato e corretto.



```

(venv)-(guillermo@vbox)-[~/proyecto_seguridad/auditoria_seguridad]
$ ./venv/bin/pip install -r requirements.txt
zsh: ./venv/bin/pip: bad interpreter: /home/guillermo/auditoria_seguridad/venv/bin/python3: no existe el fichero o es un directorio

(venv)-(guillermo@vbox)-[~/proyecto_seguridad/auditoria_seguridad]
$ ls
core  __pycache__  report.md  requirements.txt  test_cms.py  venv
main.py  report.json  reports  scanner_history.db  tests

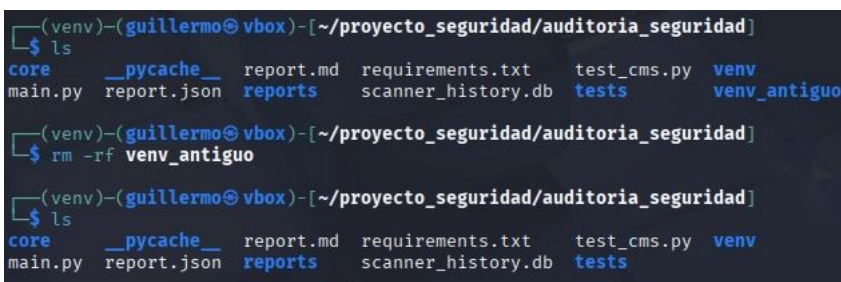
(venv)-(guillermo@vbox)-[~/proyecto_seguridad/auditoria_seguridad]
$ mv venv venv_antiguo

(venv)-(guillermo@vbox)-[~/proyecto_seguridad/auditoria_seguridad]
$ python3 -m venv venv

(venv)-(guillermo@vbox)-[~/proyecto_seguridad/auditoria_seguridad]
$ source venv/bin/activate

```

- Questo screenshot documenta la fase di pulizia e ottimizzazione dell'ambiente di lavoro. Dopo aver confermato il corretto funzionamento del nuovo ambiente virtuale, la directory di backup (``venv_antiguo``) è stata rimossa utilizzando il comando ``rm -rf``, garantendo così l'integrità del progetto. Infine, è stato eseguito un ultimo controllo tramite il comando ``ls`` per verificare che la directory radice fosse ordinata e priva di file ridondanti, concludendo formalmente la configurazione dell'ambiente.



```

(venv)-(guillermo@vbox)-[~/proyecto_seguridad/auditoria_seguridad]
$ ls
core  __pycache__  report.md  requirements.txt  test_cms.py  venv
main.py  report.json  reports  scanner_history.db  tests  venv_antiguo

(venv)-(guillermo@vbox)-[~/proyecto_seguridad/auditoria_seguridad]
$ rm -rf venv_antiguo

(venv)-(guillermo@vbox)-[~/proyecto_seguridad/auditoria_seguridad]
$ ls
core  __pycache__  report.md  requirements.txt  test_cms.py  venv
main.py  report.json  reports  scanner_history.db  tests

```

- Questo screenshot documenta la ridenominazione del file originale ``main.py`` in ``cli_main.py``. Tale modifica organizzativa consente di distinguere chiaramente tra il punto di ingresso

dell'interfaccia a riga di comando (CLI) e il resto dell'architettura del progetto, migliorando la chiarezza della struttura modulare man mano che viene integrata la nuova API.

```
(venv)-(guillermo@ vbox)-[~/proyecto_seguridad/auditoria_seguridad]
$ source venv/bin/activate

(venv)-(guillermo@ vbox)-[~/proyecto_seguridad/auditoria_seguridad]
$ pip install -r requirements.txt
Collecting beautifulsoup4==4.13.4 (from -r requirements.txt (line 1))
  Downloading beautifulsoup4-4.13.4-py3-none-any.whl.metadata (3.8 kB)
Collecting dnspython==2.7.0 (from -r requirements.txt (line 2))
  Downloading dnspython-2.7.0-py3-none-any.whl.metadata (5.8 kB)
Collecting httpcore==1.0.7 (from -r requirements.txt (line 3))
  Downloading httpcore-1.0.7-py3-none-any.whl.metadata (21 kB)
Collecting httpx==0.28.1 (from -r requirements.txt (line 4))
  Using cached httpx-0.28.1-py3-none-any.whl.metadata (7.1 kB)
Collecting requests==2.32.3 (from -r requirements.txt (line 5))
  Downloading requests-2.32.3-py3-none-any.whl.metadata (4.6 kB)
Collecting fastapi==0.115.0 (from -r requirements.txt (line 6))
  Downloading fastapi-0.115.0-py3-none-any.whl.metadata (27 kB)
```

- **Preserva la logica attuale:** dato che disponiamo già di un `main.py` perfettamente funzionante per la linea, il primo passo consiste nel rinominarlo. In questo modo, qualora fosse necessario eseguire un controllo rapido da console, la logica rimarrebbe intatta e al sicuro.

```
(venv)-(guillermo@ vbox)-[~/proyecto_seguridad/auditoria_version_2_0]
$ mv main.py cli_main.py

(venv)-(guillermo@ vbox)-[~/proyecto_seguridad/auditoria_version_2_0]
$ ls
api  cli_main.py  core  __pycache__  report.json  report.md  reports  requirements.txt  scanner_history.db  test_cms.py  tests  venv
```

- **Creazione del punto di ingresso dell'API:** creiamo un nuovo file chiamato `main.py`. Questo file funge da "cervello" del nostro server web; vi incolliamo il codice di base per inizializzare FastAPI e verificare che tutto sia installato correttamente.

```
GNU nano 8.4 main.py
from fastapi import FastAPI

app = FastAPI(title="Scanner di audit API")

@app.get("/")
def read_root():
    return {"message": "Benvenuti in Audit Engine 2.0"}

@app.get("/status")
def status():
    return {"status": "online", "version": "2.0"}
```

- Questa immagine mostra l'avvio del server web tramite Uvicorn. Eseguendo il comando `uvicorn main:app --reload` si avvia l'API; il parametro `--reload` è fondamentale, poiché consente al server di rilevare automaticamente le modifiche al codice e riavviarsi senza intervento manuale. Il messaggio di conferma "Application startup complete" indica che l'API è operativa e in attesa di richieste all'indirizzo `[http://127.0.0.1:8000](http://127.0.0.1:8000)`.

```
(venv)-(guillermo@ vbox)-[~/proyecto_seguridad/auditoria_version_2_0]
$ uvicorn main:app --reload
INFO: Will watch for changes in these directories: ['/home/guillermo/proyecto_seguridad/auditoria_version_2_0']
INFO: Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO: Started reloader process [359474] using StatReload
INFO: Started server process [359476]
INFO: Waiting for application startup.
INFO: Application startup complete.
```

- Questo codice inizializza l'API utilizzando **FastAPI** e impiega **Pydantic** per verificare che le richieste contengano un URL. Definisce due route: ``/`` per una risposta di base e ``POST /audit``,

che riceve l'URL e conferma la preparazione della scansione, ponendo le basi per l'integrazione della logica di audit.

```
GNU nano 8.4 main.py
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI(title="Scanner di audit API")

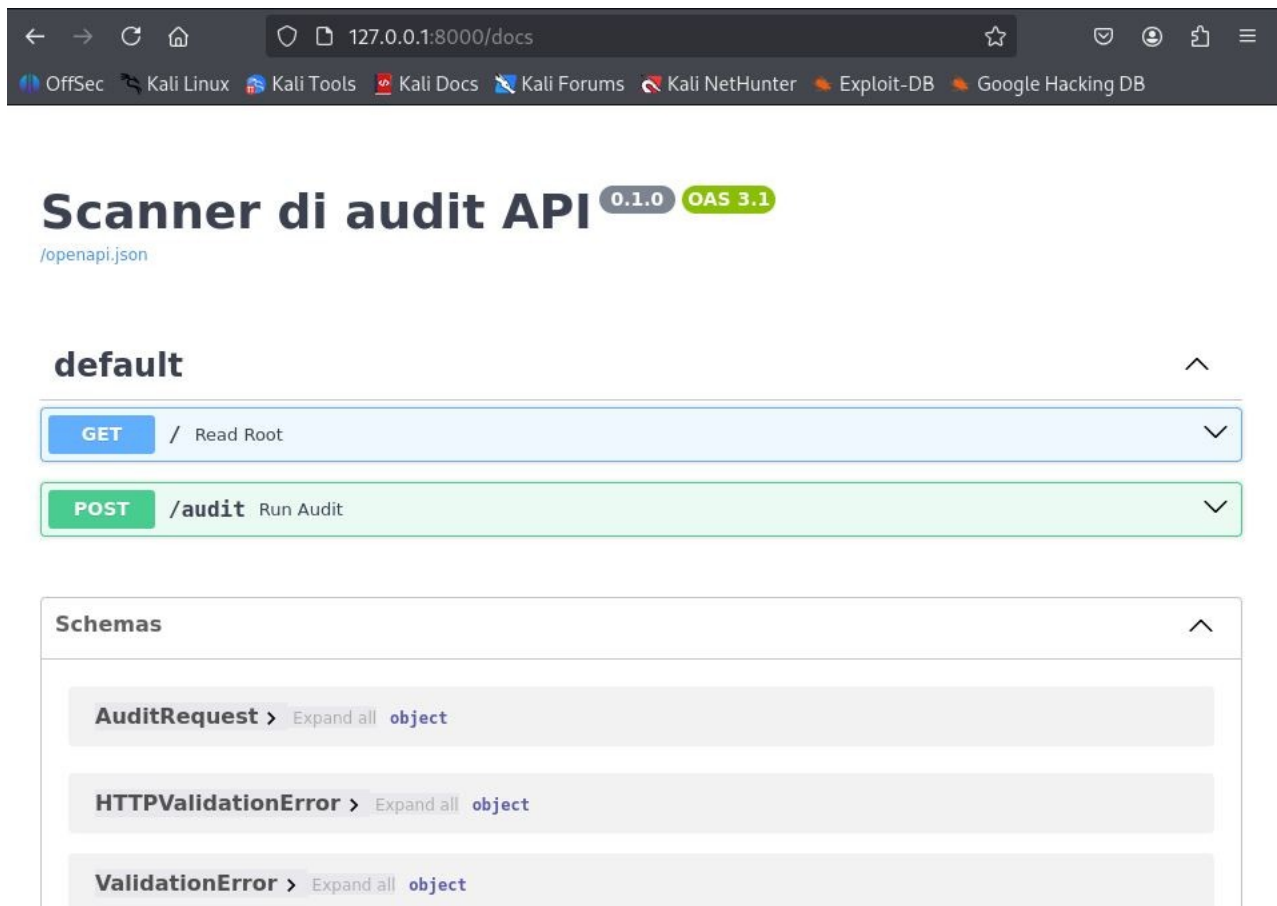
# Definiamo le modalità di arrivo dei dati (lo schema)
class AuditRequest(BaseModel):
    url:str

@app.get("/")
def read_root():
    return {"message": "Benvenuti in Audit Engine 2.0"}

# Nuovo endpoint per ricevere la richiesta di audit
@app.post("/audit")
def run_audit(request: AuditRequest):
    #In futuro, qui verrà inserita la logica dello scanner
    return {"status": "Ricevuto", "message": f"Preparazione dell'audit per: {request.url}"}
```

- Questo screenshot mostra il server Uvicorn in esecuzione e in ascolto sulla porta 8000. I log indicano l'accesso riuscito alla route `/docs`, confermando che il sistema è operativo e sta caricando la documentazione automatica dell'API.
- E se apriamo il browser, possiamo visualizzarla.

```
(venv)-(guillermo@vbox)-[~/proyecto_seguridad/auditoria_version_2_0]
$ uvicorn main:app --host 0.0.0.0 --port 8000 --reload
INFO: Will watch for changes in these directories: ['/home/guillermo/proyecto_seguridad/auditoria_version_2_0']
INFO: Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
INFO: Started reloader process [358784] using StatReload
INFO: Started server process [358786]
INFO: Waiting for application startup.
INFO: Application startup complete.
INFO: 127.0.0.1:56226 - "GET /docs HTTP/1.1" 200 OK
INFO: 127.0.0.1:56226 - "GET /openapi.json HTTP/1.1" 200 OK
```



- Questo codice integra la logica di auditing originale in un'API realizzata con FastAPI. All'avvio dell'applicazione, la funzione `init_db` garantisce la predisposizione del database. L'endpoint `POST /audit` funge da controller principale: riceve l'URL, esegue la scansione tramite `SecurityScanner`, genera automaticamente report nei formati **MD** e **JSON** e restituisce i risultati finali all'utente.

```

GNU nano 8.4 main.py
from fastapi import FastAPI
from pydantic import BaseModel
# Importiamo le tue classi e funzioni originali
from core.scanner import SecurityScanner
from reports.generator import ReportGenerator
from core.database import init_db

app = FastAPI(title="Scanner di audit API")

# Definiamo il modello dati per ricevere l'URL
class AuditRequest(BaseModel):
    url:str

@app.on_event("startup")
def startup_event():
    # Ciò garantisce che il database sia pronto all'avvio
    init_db()

@app.get("/")
def read_root():
    return {"message": "Benvenuti in Audit Engine 2.0 - API attiva"}

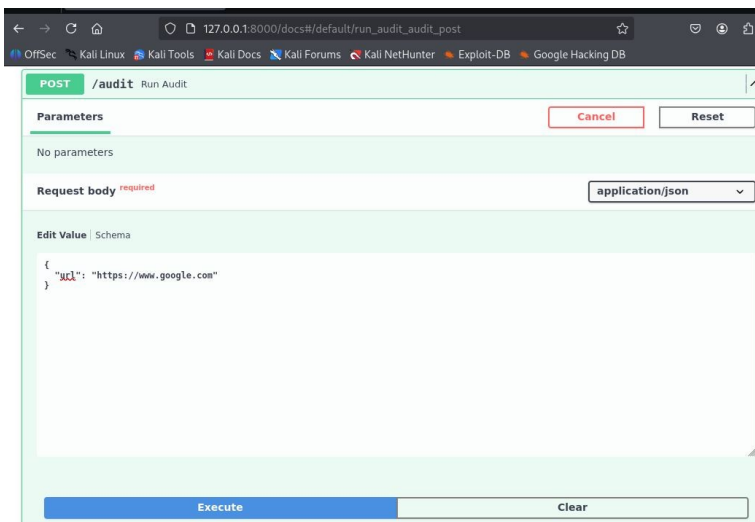
# Nuovo endpoint per ricevere la richiesta di audit
@app.post("/audit")
def run_audit(request: AuditRequest):
    # 1. Eseguimo la scansione (utilizzando la tua logica originale)
    scanner = SecurityScanner(request.url)
    risultati = scanner.run_audit()

    # 2. Generiamo i report (proprio come hai fatto tu tramite CLI)
    reporter = ReportGenerator(risultati)
    reporter.save("md")
    reporter.save("json")

    # 3. Restituiamo i risultati a chi ha effettuato la richiesta
    return {"status": "completato", "url_auditada": request.url, "risultati": risultati}
  
```

- Il sistema offre un'interfaccia di documentazione interattiva che semplifica notevolmente l'interazione con le API. Tramite questa interfaccia web, l'utente può inserire facilmente l'URL di destinazione (ad esempio "<https://www.google.com>") nel corpo della richiesta JSON per l'endpoint `/audit` ed eseguire la scansione direttamente dal browser.
- All'invio della richiesta, i log del server nel terminale confermano che il backend sta elaborando correttamente la scansione. La console riporta i dettagli relativi alla ricezione della richiesta e alla generazione automatica dei report risultanti nei formati Markdown e JSON, concludendo il processo con un codice di stato **HTTP 200**, a conferma dell'esito positivo dell'operazione.

```
(venv)-(guillermo@vbox)-[~/proyecto_seguridad/auditoria_version_2_0]
$ uvicorn main:app --host 0.0.0.0 --port 8000 --reload
INFO: Will watch for changes in these directories: ['/home/guillermo/proyecto_seguridad/auditoria_version_2_0']
INFO: Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
INFO: Started reload process [357095] using StatReload
INFO: Started server process [357097]
INFO: Waiting for application startup.
INFO: Application startup complete.
INFO: 127.0.0.1:37032 - "GET /docs HTTP/1.1" 200 OK
INFO: 127.0.0.1:37032 - "GET /openapi.json HTTP/1.1" 200 OK
[DEBUG] Formate ricevuto: 'md'
[+] Report Markdown generato: report.md
[DEBUG] Formate ricevuto: 'json'
[+] Report JSON generato: report.json
INFO: 127.0.0.1:55334 - "POST /audit HTTP/1.1" 200 OK
```



- Creiamo la cartella API, spostiamo il file `main.py` nella nuova cartella rinominandolo e lo apriamo utilizzando questi comandi:

```
(venv)-(guillermo@vbox)-[~/proyecto_seguridad/auditoria_version_2_0]
$ mkdir api
(venv)-(guillermo@vbox)-[~/proyecto_seguridad/auditoria_version_2_0]
$ mv main.py api/routes.py
(venv)-(guillermo@vbox)-[~/proyecto_seguridad/auditoria_version_2_0]
$ nano main.py
```

Nota: Creiamo il file `__init__.py` utilizzando il comando `touch`.

- **Integrazione:** Passando ``args.url`` da ``main``, colleghiamo l'input della CLI alla logica del database.

```
GNU nano 8.4 main.py
import uvicorn
from api.routes import app

if __name__ == "__main__":
    uvicorn.run(app, host="0.0.0.0", port=8000)
```

- Esaminiamo la struttura delle cartelle.

¿Perché questa soluzione è migliore?

- **Root pulita:** la nostra directory radice ora contiene solo le cartelle di lavoro e ``main.py``, che funge da punto di avvio per l'intero sistema.
- **Separazione netta:** se in futuro decidessimo di realizzare un'applicazione desktop, basterebbe creare una cartella ``gui/``; la logica contenuta in ``core/`` rimarrebbe intatta e non interferirebbe con le API.
- **Professionalità:** stiamo applicando il principio della "Separazione delle responsabilità" (Separation of Concerns).

```
(venv)-(guillermo@vbox)-[~/proyecto_seguridad/auditoria_version_2_0]
$ tree -I "__pycache__|venv|.git"
.
├── api
│   ├── __init__.py
│   └── routes.py
├── core
│   ├── cms_detector.py
│   ├── database.py
│   ├── __init__.py
│   ├── scanner.py
│   ├── scoring.py
│   └── utils.py
├── main.py
├── report.json
├── report.md
├── reports
│   ├── generator.py
│   ├── __init__.py
│   ├── requirements.txt
│   ├── scanner_history.db
│   └── test_cms.py
├── tests
│   ├── __init__.py
│   └── test_scanner.py
└── 5 directories, 18 files
```

- **L'alternativa di sicurezza (fail-safe):** riscontriamo errori dovuti a problemi di rete derivanti dalle modalità con cui il sistema operativo gestisce i socket.

```
(venv)-(guillermo@vbox)-[~/proyecto_seguridad/auditoria_version_2_0]
$ python3 main.py --url https://asociaciondigitalcanaria.es
INFO: Started server process [352015]
INFO: Waiting for application startup.
INFO: Application startup complete.
ERROR: [Errno -2] Name or service not known
INFO: Waiting for application shutdown.
INFO: Application shutdown complete.
```

- **Integrazione:** Passando ``args.url`` da ``main``, colleghiamo l'input della CLI alla logica del database.
- Il messaggio "Application startup complete" funge da conferma definitiva del passaggio da uno script autonomo a un'applicazione web strutturata, modulare e professionale.
 - **Struttura professionale:** la logica è isolata in ``core/``, i report in ``reports/`` e il livello web in ``api/``.
 - **Funzionalità:** il server si avvia senza errori di rete.
 - **Scalabilità:** se volessimo aggiungere nuove funzionalità, ora sappiamo esattamente quale file modificare senza compromettere il sistema.

```
(venv)-(guillermo@vbox)-[~/proyecto_seguridad/auditoria_version_2_0]
$ uvicorn api.routes:app --host 0.0.0.0 --port 8000 --reload
INFO: Will watch for changes in these directories: ['/home/guillermo/proyecto_seguridad/auditoria_version_2_0']
INFO: Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
INFO: Started reloader process [353891] using StatReload
INFO: Started server process [353893]
INFO: Waiting for application startup.
```